

Diagrama 1 - Arquitectura de software (escalar y vectorial)

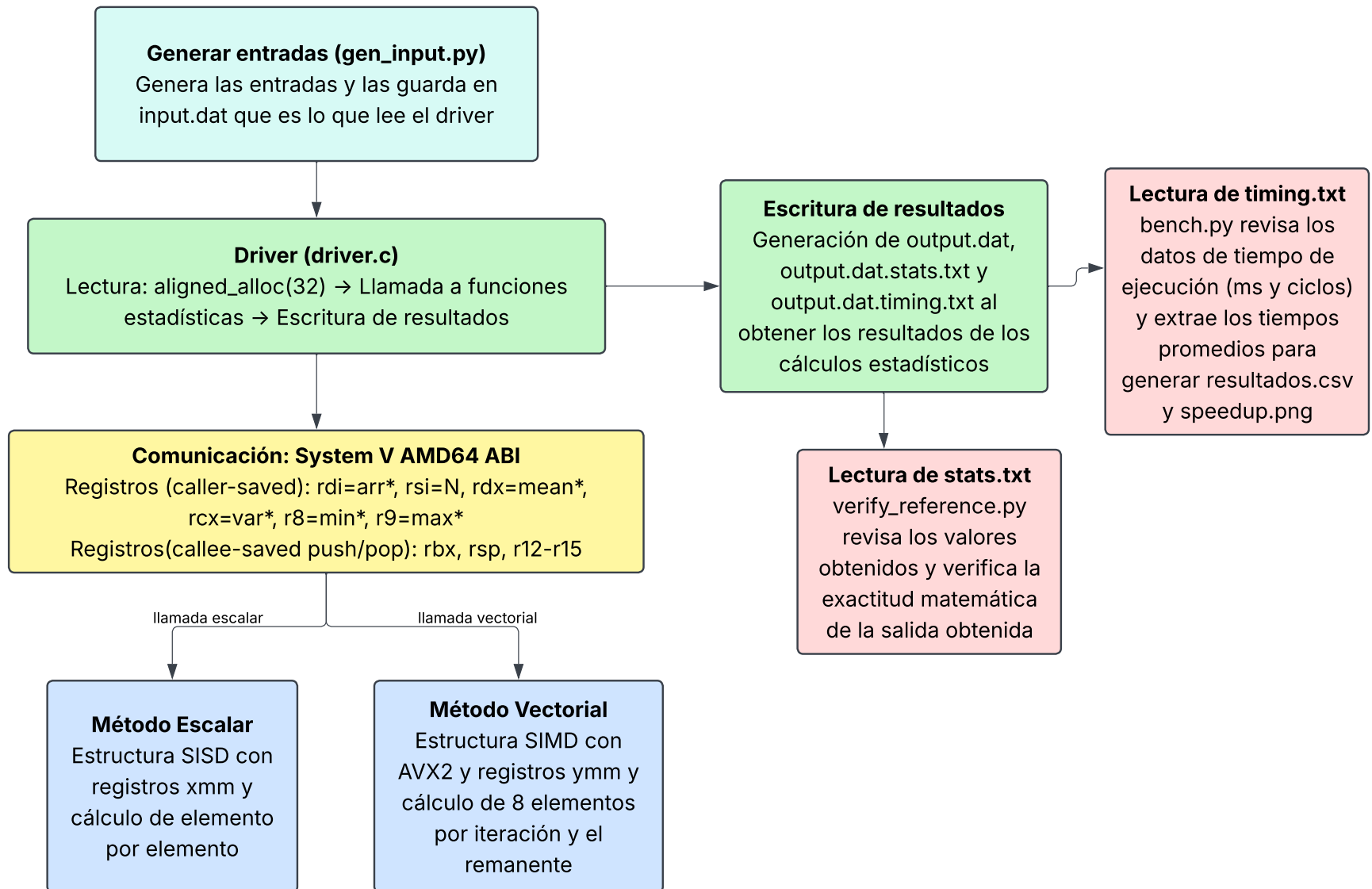


Diagrama 2 - Flujo de control de compute_stats: version escalar

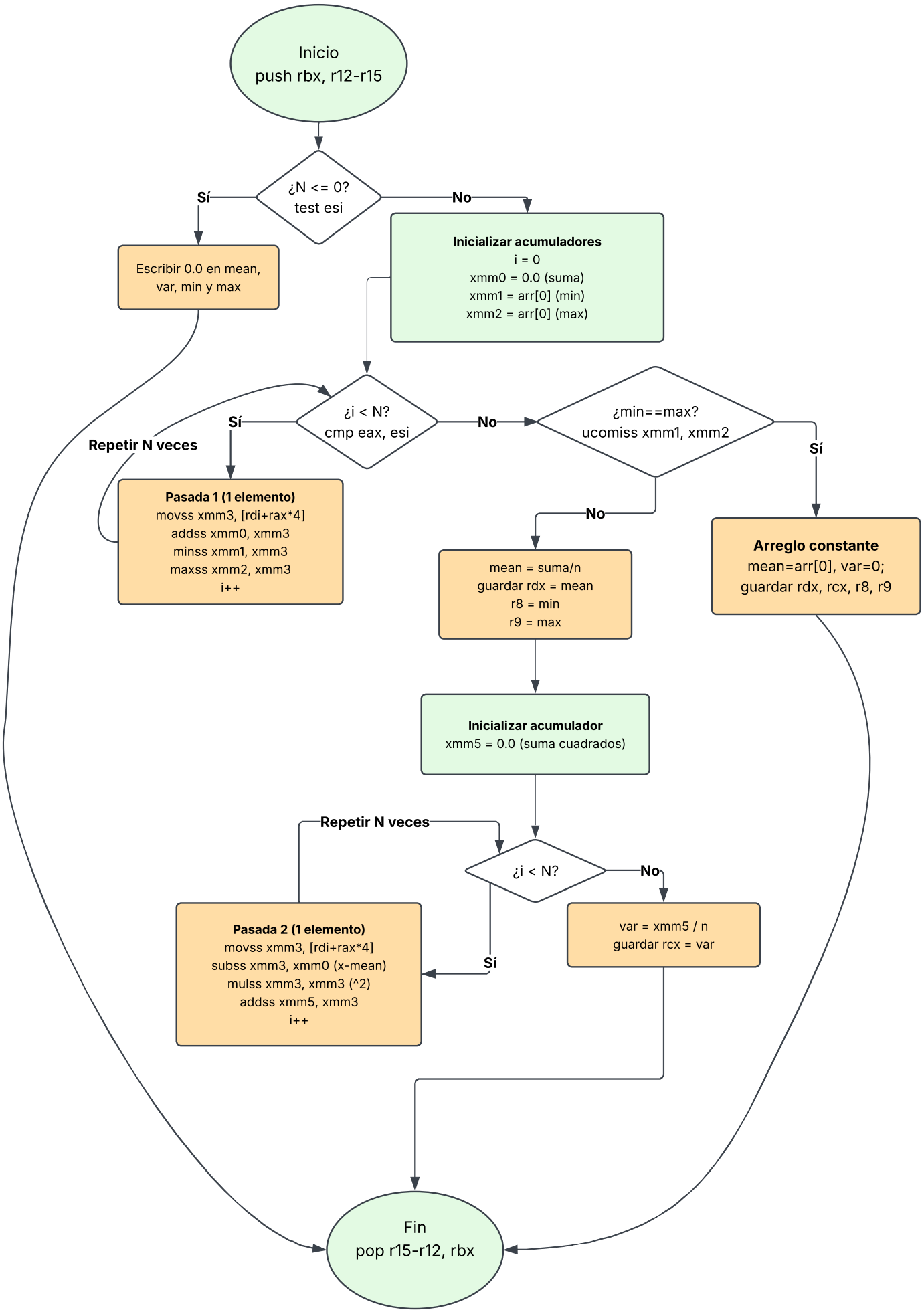


Diagrama 3 - Flujo de control de compute_stats: version vectorial AVX2

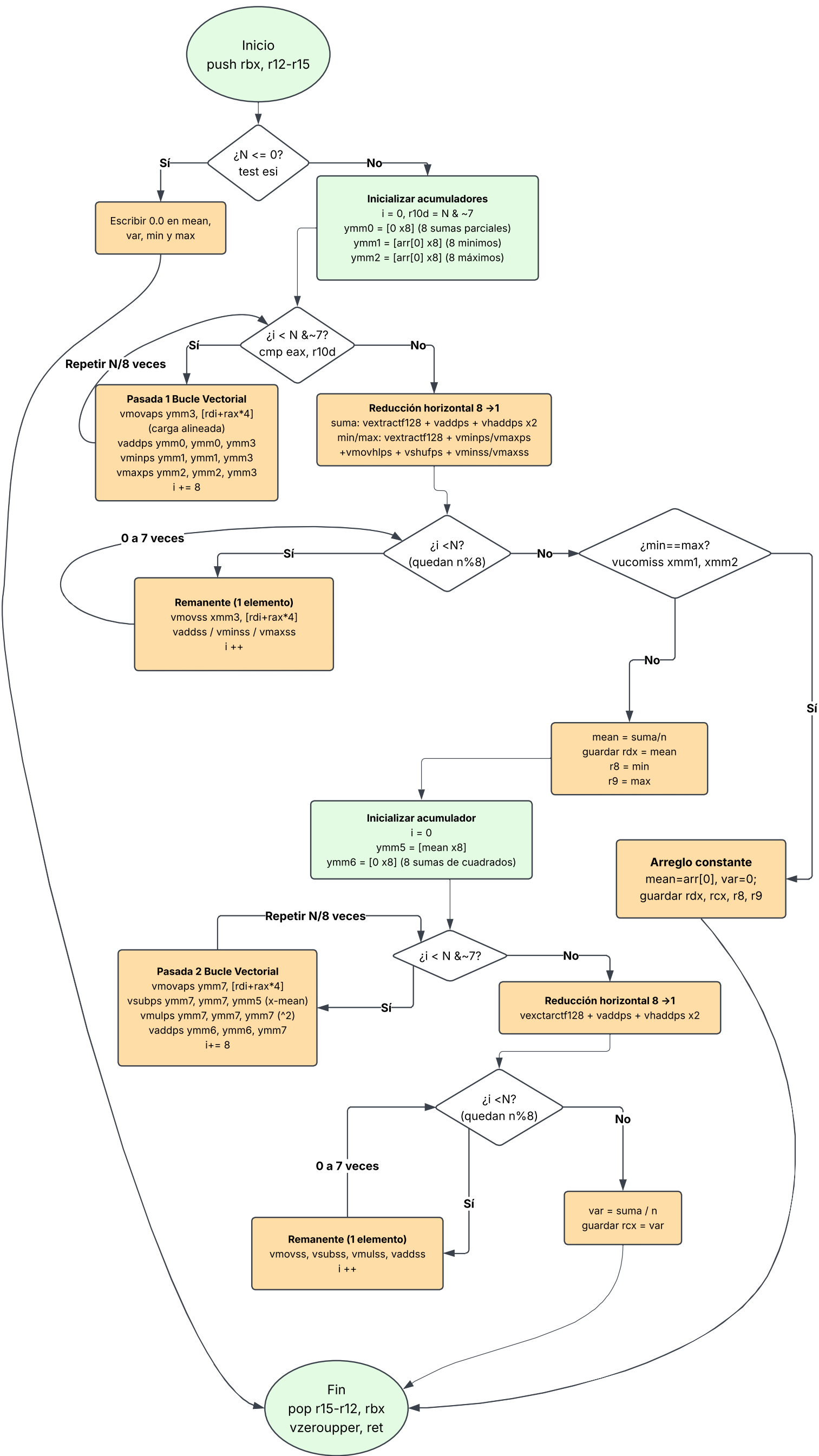


Diagrama 4 - Asignacion de registros en compute_stats (kernel critico)

VERSION VECTORIAL AVX2 (stats_vector.asm)										
Registro	Entrada (al llamar)	Inicializacion (A2)	Pasada 1 bucle vectorial	Reduccion 1 8 a 1	Remanente 1	Media y guardado (A6, min != max)	Arreglo constante (A6b, min == max)	Pasada 2 init + bucle vect.	Reduccion 2 + remanente 2	Varianza y salida
rdi	arr	arr	arr (base)	-	arr (base)	-	-	arr (base)	arr (base)	-
esi	n	n	-	-	n (limite)	n a float	-	-	n (limite)	-
rdx	mean*	-	-	-	-	escribe mean	escribe mean = arr[0]	-	-	-
rcx	var*	-	-	-	-	-	escribe var = 0.0	-	-	escribe var
r8	min*	-	-	-	-	escribe min	escribe min	-	-	-
r9	max*	-	-	-	-	escribe max	escribe max	-	-	-
eax (rax)	-	i = 0	i (+8)	i	i (+1)	-	-	i = 0, luego +8	i (+1)	-
r10d	-	n & ~7	limite vectorial	-	-	-	-	limite vectorial	-	-
ymm0 / xmm0	-	[0 x8]	8 sumas parciales	xmm0[0] = suma de 8 carriles	+ arr[i] (suma total)	mean	0.0 (valor de var)	mean (fuente broadcast)	mean (para x - mean)	-
ymm1 / xmm1	-	[arr[0] x8]	8 minimos parciales	xmm1[0] = min de 8 carriles	min(., arr[i])	min final (vucomiss con max)	min = max = arr[0] (se escribe como mean)	-	-	-
ymm2 / xmm2	-	[arr[0] x8]	8 maximos parciales	xmm2[0] = max de 8 carriles	max(., arr[i])	max final (vucomiss con min)	max	-	-	-
ymm3 / xmm3	-	-	arr[i..i+7]	temporal (mitad alta, shuffle)	arr[i]	-	-	-	-	-
xmm4	-	-	-	-	-	(float) n	(float) n (no se usa)	-	-	divisor n
ymm5	-	-	-	-	-	-	-	[mean x8]	-	-
ymm6 / xmm6	-	-	-	-	-	-	-	8 sumas de (x - mean)^2	xmm6[0] = suma total	var
ymm7 / xmm7	-	-	-	-	-	-	-	x, x - mean, (x - mean)^2	temporal / x	-
rbx, r12-r15	push (pila)	-	-	-	-	-	pop (salta pasada 2)	-	-	pop

VERSION ESCALAR (stats_scalar.asm)							
Registro	Entrada (al llamar)	Inicializacion (C2)	Pasada 1 (1 elemento)	Media y guardado (C4, min != max)	Arreglo constante (C4b, min == max)	Pasada 2 (1 elemento)	Varianza y salida
rdi	arr	arr	arr (base)	-	-	arr (base)	-
esi	n	n	n (limite)	n a float	-	n (limite)	-
rdx / rcx / r8 / r9	mean* / var* / min* / max*	-	-	escribe mean, min, max	escribe mean = arr[0], var = 0.0, min, max	-	escribe var
eax (rax)	-	i = 0	i (+1)	-	-	i = 0, luego +1	-
xmm0	-	0.0	suma	mean	0.0 (valor de var)	mean	-
xmm1	-	arr[0]	min	min final (ucomiss con max)	min = max = arr[0] (se escribe como mean)	-	-
xmm2	-	arr[0]	max	max final (ucomiss con min)	max	-	-
xmm3	-	-	arr[i]	-	-	x, x - mean, (x - mean)^2	-
xmm4	-	-	-	(float) n	(float) n (no se usa)	-	divisor n
xmm5	-	-	-	-	-	suma de (x - mean)^2	var
rbx, r12-r15	push (pila)	-	-	-	pop (salta pasada 2)	-	pop

Notas

- * ymm = 256 bits = 8 floats. xmm = los 128 bits bajos del mismo registro. "xmm0[0]" = carril 0 (el float mas bajo).
- * Se usa r10d (no ecx) como limite vectorial porque rcx contiene el puntero var*. r10 es caller-saved: se puede usar sin guardarlo.
- * No se llama a sum_array, asi que ningun argumento se destruye: rdi, esi, rdx, rcx, r8 y r9 conservan su valor todo el kernel.
- * rbx y r12-r15 se guardan con push y se restauran con pop (esqueleto base), aunque este kernel no los usa.
- * A6b / C4b: tras la pasada 1 se compara min con max (vucomiss / ucomiss). Si son iguales, todos los elementos lo son: se escriben mean = arr[0] y var = 0.0 exactos y se salta la pasada 2. Si alguno es NaN (PF = 1) se sigue el camino general.
- * Los colores de las columnas corresponden a las etapas de los Diagramas 2 y 3; en rojo claro, la rama de arreglo constante.

Diagrama 5 - Mapa de memoria: alineacion y avance del puntero

Alineacion de las direcciones base (valores reales de la sesion de GDB, N = 16)			
Arreglo	Direccion base	Ultimo byte en binario	Verificacion
in (rdi)	0x55555555a4c0	0xc0 = 110 0 0000	5 bits bajos en 0: multiplo de 32 (0xc0 = 192 = 6 x 32)
out (rsi)	0x555555559480	0x80 = 100 0 0000	5 bits bajos en 0: multiplo de 32 (0x80 = 128 = 4 x 32)
Regla: una direccion es multiplo de 32 = 2 ⁵ si sus 5 bits menos significativos son 0. La garantiza el driver: aligned_alloc(32, tamaño redondeado a multiplo de 32). El relleno se pone en 0 con memset.			

Arreglo in (ejemplo ilustrativo N = 21, misma base que en GDB: 0x55555555a4c0)																								
Elemento	in[0]	in[1]	in[2]	in[3]	in[4]	in[5]	in[6]	in[7]	in[8]	in[9]	in[10]	in[11]	in[12]	in[13]	in[14]	in[15]	in[16]	in[17]	in[18]	in[19]	in[20]	relleno	relleno	relleno
Offset (bytes)	+0	+4	+8	+12	+16	+20	+24	+28	+32	+36	+40	+44	+48	+52	+56	+60	+64	+68	+72	+76	+80	+84	+88	+92
Direccion	0x55555555a4c0 (bloque de 32 B, multiplo de 32)								0x55555555a4e0 (bloque de 32 B, multiplo de 32)								0x55555555a500 (bloque de 32 B, multiplo de 32)							
Linea de cache (64 B)	linea de cache k: 64 bytes = 16 floats = 2 cargas YMM																linea k+1 (primera mitad)							
Version VECTORIAL	iteracion 1: vmovaps ymm, [rdi + 0] (8 floats de una vez)								iteracion 2: vmovaps ymm, [rdi + 32] (elementos 8 a 15, puntero +32 B)								rem. 1 vmovss	rem. 2 vmovss	rem. 3 vmovss	rem. 4 vmovss	rem. 5 vmovss	no se lee (i = n, fin)		
Version ESCALAR	it. 1 movss	it. 2 movss	it. 3 movss	it. 4 movss	it. 5 movss	it. 6 movss	it. 7 movss	it. 8 movss	it. 9 movss	it. 10 movss	it. 11 movss	it. 12 movss	it. 13 movss	it. 14 movss	it. 15 movss	it. 16 movss	it. 17 movss	it. 18 movss	it. 19 movss	it. 20 movss	it. 21 movss	no se lee		

Avance del puntero: direccion = rdi + rax * 4			
Bucle	Incremento de i (rax)	Avance en memoria	Direcciones visitadas (in, N = 21)
Escalar	add 1 (inc eax)	+4 bytes (1 float)	+0, +4, +8, ..., +80 (21 accesos)
Vectorial: bucle principal	add 8	+32 bytes (8 floats)	+0, +32 (2 accesos alineados con vmovaps)
Vectorial: remanente	add 1 (inc eax)	+4 bytes (1 float)	+64, +68, +72, +76, +80 (5 accesos con vmovss)

Por que importa la alineacion

- * vmovaps exige una direccion multiplo de 32: si no lo es, el procesador genera una excepcion (el programa termina con Segmentation fault).
- * Base alineada + avance de 32 bytes: todas las cargas del bucle principal quedan alineadas.
- * Una linea de cache mide 64 bytes (multiplo de 32): una carga alineada de 32 bytes nunca queda partida entre dos lineas de cache.
- * El remanente usa vmovss (4 bytes), que no tiene requisito de alineacion.

Por que el remanente no carga 8 floats del ultimo bloque

- * El relleno vale 0: sumarlo no cambia la suma, pero si el arreglo es positivo, min seria 0 (incorrecto) y la media/varianza usarian datos que no existen.
- * En normalize_array se escribiria fuera de los n elementos pedidos. Por eso los sobrantes se procesan de uno en uno.
- * El arreglo out tiene exactamente la misma disposicion (base 0x555555559480) y se recorre con los mismos offsets.